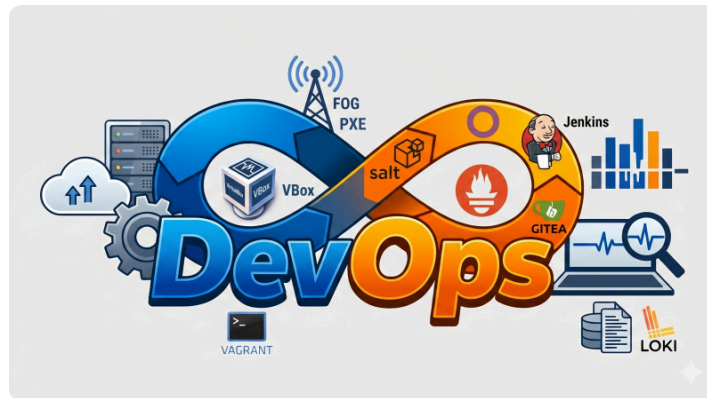


DevOps. Automatización y monitorización de infraestructuras

Unidad 05.

Automatización y monitorización en Odoo



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 05. Automatización y monitorización en Odoo	2
1. 🧠 Introducción	2
2. 🏢 La aplicación a desplegar: Odoo ERP	2
2.1 Modelos de despliegue: El estándar Docker	3
2.2 Optimización de rendimiento y workers (corrección importante)	3
3. 🏗️ Infraestructura como Código (IaC) con Terraform	4
3.1 Control granular quirúrgico	4
4. 🧴 Gestión de configuración escalable con SaltStack	4
4.1 Conceptos fundamentales de SaltProject	5
5. 👁️ Observando el sistema ERP Odoo	7
5.1 El puente entre infraestructura y negocio (Whitebox vs. Blackbox)	7
5.2 El stack tecnológico de métricas	7
5.3 Logs centralizados: la magia de Alloy y Loki (solo en la versión completa)	8
6. 📊 Métricas críticas y cultura de fiabilidad (SRE)	9
6.1 Disponibilidad del servicio web (probe_success)	9
6.2 Métricas de adopción del negocio (odoo_users_total, odoo_sales_orders_total)	9
6.3 Latencia de las consultas a la base de datos (odoo_query_duration_seconds)	9
6.4 Saturación de memoria en contenedores (container_memory_working_set_bytes)	10
7. ★ Buenas prácticas en automatización y monitorización	10
7.1 Idempotencia: ejecuta el despliegue tantas veces como quieras	10
7.2 Mitigación de la fatiga de alertas (alert fatigue)	10
7.3 Desacoplamiento de datos: los volúmenes salvan vidas	10
7.4 Principio de mínimo privilegio: no ejecutes como root	11
7.5 Automatización no interactiva (modo desatendido)	11
7.6 Seguridad en la red y cifrado	11
7.7 Monitorización de los propios sistemas de monitorización	11
7.8 Documentación como código	11
8. 📚 Webgrafía.	12

UNIDAD 05. AUTOMATIZACIÓN Y MONITORIZACIÓN EN ODOO

1. INTRODUCCIÓN

En las unidades anteriores, sentamos las bases de la infraestructura como código (IaC) y el despliegue de sistemas operativos. Ya sabes cómo encender servidores y orquestar contenedores y algunos ligeros toques de observabilidad. En esta unidad vamos a dar algunos pasos más con un sistemas más complejo y extenso.

El modelo mental del arquitecto

Para entender el ecosistema de 11 contenedores que vamos a orquestar, imagina que estás diseñando el centro de mando de una fábrica inteligente automatizada:

1. **El terreno y el edificio (Terraform):** Define cuántas naves industriales necesitas, sus almacenes persistent.es (volúmenes) y los pasillos de comunicación (red bridge LabCEFIRE).
2. **El jefe de planta (Salt Master):** Da las órdenes por un megáfono cifrado Los Minions son los operarios en cada nave que aplican el manual de procedimientos.
3. **Las líneas de producción (Odoo + PostgreSQL):** Son el corazón del negocio. Tenemos dos líneas de montaje gemelas para asegurar que la fábrica nunca deje de facturar.
4. **Las cámaras de seguridad (Prometheus + Exporters):** Sensores instalados en toda la fábrica que anotan la temperatura y el ritmo de trabajo cada 15 segundos.
5. **El videowall (Grafana):** Es la pantalla gigante del director, donde los miles de números crudos se traducen en gráficos de colores.
6. **El perito y el archivo (Alloy + Loki):** Si ocurre un accidente, Alloy recoge todos los registros de la planta y Loki los archiva cronológicamente para investigar la causa raíz.

Nota sobre versiones del laboratorio: El documento describe un ecosistema completo de **11 contenedores** (incluyendo Loki y Alloy). No obstante, se incluye una **variante de “bajos recursos”** (sin Loki/Alloy y con un solo Odoo) para entornos con limitaciones de memoria (≤ 8 GB RAM). Ambas versiones comparten los principios de automatización y monitorización que se explican aquí.

2. LA APLICACIÓN A DESPLEGAR: ODOO ERP

Odoo es un ERP (Enterprise Resource Planning) de código abierto con un modelo de negocio *open source + servicios en la nube*. Su principal fortaleza son más de 70 aplicaciones oficiales (gratuitas y de pago) y miles de módulos comunitarios que cubren prácticamente todas las áreas de una empresa.

Arquitectura interna

- **Backend (Python):** Aloja la lógica de negocio y se comunica con la base de datos y el frontend.
- **PostgreSQL:** Motor de persistencia de datos. **Cada instancia de Odoo debe tener su**

propia base de datos para evitar conflictos de esquema.

- **Frontend (JavaScript / Owl Framework):** Interfaz web moderna que se comunica con el backend mediante **JSON-RPC**.
- **Servidor web integrado (Werkzeug):** Orquesta las peticiones HTTP y gestiona los workers.

2.1 Modelos de despliegue: El estándar Docker

Aunque existen soluciones cloud, el despliegue on-premise o self-hosted (frecuentemente con Docker) proporciona un control absoluto sobre la seguridad de los datos y las personalizaciones de código. Docker se ha consolidado como el estándar de oro para Odoo porque encapsula todas las dependencias críticas –como wkhtmltopdf para exportar informes o librerías específicas de Python– en imágenes inmutables.

En nuestro laboratorio, cada instancia de Odoo (odoo-minion-1, odoo-minion-2) tiene su propia base de datos dedicada (odoo-pg1, odoo-pg2). Esto ejemplifica el patrón DevOps de desacoplar el estado de la aplicación: si un contenedor web es destruido, los datos financieros permanecen a salvo y persistentes gracias a los volúmenes Docker.

2.2 Optimización de rendimiento y workers (corrección importante)

Odoo utiliza un modelo de procesos paralelos denominados workers para manejar la concurrencia. En un entorno de producción, la configuración debe ajustarse a los recursos reales:

- **Fórmula de workers (web):** (número de núcleos de CPU) * 2 + 1.
 - Por ejemplo, en un servidor de 4 núcleos físicos, la configuración óptima sería de 9 workers.
 - El +1 asegura que siempre haya un proceso disponible para peticiones ligeras o tareas programadas (cron) mientras los demás procesan carga pesada.
- **Workers para tareas programadas (cron):** Pueden configurarse por separado con `cron_workers`. Un valor habitual es igual al número de núcleos (4 en el ejemplo).
- **Consumo de RAM:** Cada worker consume típicamente entre 256 MB y 512 MB (no 1 GB, a menos que los módulos sean extremadamente pesados).
- **Límites estrictos:** En los archivos de configuración de Odoo se definen `limit_memory_hard` y `limit_memory_soft` (por ejemplo, 2560 MB). Si un contenedor supera ese umbral, el OOM Killer del kernel de Linux sacrifica únicamente ese contenedor, evitando que colapse todo el servidor físico.

Ejemplo práctico del laboratorio

En el despliegue de bajos recursos, el contenedor Odoo tiene asignada 1024 MB de RAM (`memory = 1024` en Terraform) y se configura con un único worker (por simplicidad). Para un entorno de producción se recomienda seguir la fórmula anterior y ajustar los límites de memoria en el archivo `odoo.conf`.

3. INFRAESTRUCTURA COMO CÓDIGO (IaC) CON TERRAFORM

Mientras que herramientas como Vagrant se enfocan en entornos de desarrollo local, y Docker Compose es útil para levantar servicios acoplados, **Terraform** nos permite gestionar infraestructuras complejas, distribuidas y escalables a nivel de producción.

En este ecosistema, Terraform no se limita a "encender" contenedores; orquesta una topología de 11 nodos interconectados (o una versión reducida de 7 nodos para bajos recursos). El archivo **terraform.tf** actúa como el plano de la fábrica.

3.1 Control granular quirúrgico

- **Enlace de red y direccionamiento fijo:** Conectamos los contenedores a la red bridge LabCEFIRE (172.28.0.0/24) y asignamos IPs estáticas a cada nodo (ej. .57 y .58 para Odoo, .50 para Salt Master). Esto garantiza una comunicación predecible y evita depender de la resolución DNS dinámica de Docker (que puede fallar en reinicios).
- **Persistencia desacoplada (volúmenes):** Definimos volúmenes físicos en el host para PostgreSQL, Prometheus, Grafana y (en la versión completa) Loki. Si un contenedor muere o se actualiza, los datos financieros y el historial de métricas no desaparecen. Los volúmenes se nombran con el prefijo ud05_ para evitar colisiones.
- **Inyección de archivos con upload:** En lugar de usar montajes de carpetas compartidas (que pueden tener problemas de permisos), utilizamos la directiva upload de Terraform. Esta copia físicamente los archivos de configuración (por ejemplo, prometheus-odoo.yml, config.alloy, odoo-exporter.py) y los scripts de arranque (startup-odoo.sh, etc.) directamente dentro del sistema de archivos del contenedor en el momento de su creación.
- **Bootstrapping combinado (síncrono + asíncrono):** Terraform configura el entrypoint para que ejecute un script personalizado (ej. startup-odoo.sh). Dentro de ese script, las tareas críticas (esperar a PostgreSQL, lanzar Odoo) se hacen en primer plano de forma síncrona, mientras que las tareas no bloqueantes (instalación de Salt Minion, comprobación periódica de la base de datos para lanzar el exporter) se ejecutan en segundo plano (asíncronas). Esto acelera el arranque del servicio principal sin dejar de lado la configuración completa.

Limitación importante de Terraform con Docker

Terraform no puede reiniciar un proceso si este falla dentro del contenedor (no es un orquestador de procesos). Por eso los scripts de arranque incluyen bucles de espera y healthchecks propios. Además, cada vez que se ejecuta terraform apply se re-suben los archivos binarios (como node_exporter de ~20 MB), lo que puede ralentizar el despliegue. En entornos de producción se recomienda construir imágenes personalizadas (Dockerfile) en lugar de inyectar binarios en cada aplicación.

4. GESTIÓN DE CONFIGURACIÓN ESCALABLE CON SALTSTACK

A diferencia de herramientas que se conectan secuencialmente por SSH (como Ansible), SaltStack opera bajo una arquitectura Master-Minion utilizando un bus de mensajes (ZeroMQ). Trata a los servidores como "ganado" (con instrucciones imperativas) y no como "mascotas",

permitiendo gobernar miles de nodos en milisegundos.

4.1 Conceptos fundamentales de SaltProject

- **Confianza criptográfica (Zero Trust):** Cada nodo administrado (Minion) genera una llave RSA y la envía al Master. El servidor no envía ninguna orden hasta que el administrador acepte explícitamente la llave (**salt-key -A**). En nuestro laboratorio, el Master (ud05-salt-master) acepta automáticamente las llaves cada 15 segundos mediante un bucle en segundo plano (por simplicidad didáctica, aunque en producción se haría con políticas más estrictas).
- **Grains y pillars:** la inteligencia de la configuración
 - **Grains:** Datos estáticos sobre el Minion (SO, memoria, IP). Se usan para aplicar configuraciones condicionales, por ejemplo: instalar un paquete solo si el nodo tiene Ubuntu.
 - **Pillars:** Información dinámica y sensible (contraseñas, tokens). En nuestro laboratorio, las contraseñas de PostgreSQL se inyectan mediante variables de entorno desde Terraform, no a través de Pillars, porque los contenedores de Odoo no tienen instalado el servicio Salt Minion (solo el Master tiene Salt). Esta es una limitación importante: el laboratorio no ejemplifica el uso real de Pillars.
- **Idempotencia mediante States (.sls):** Los archivos YAML describen el estado final deseado. Si ordenamos a Salt instalar dependencias de Python, el motor comprueba primero si ya están instaladas; si es así, no hace nada. Ejecutar un estado diez veces seguidas es seguro y no causa daño colateral.
- **Ejecución ad-hoc con cmd.run y test.ping:**

Salt permite ejecutar comandos arbitrarios en los Minions de forma inmediata sin necesidad de escribir un estado. Esto es muy útil para diagnósticos y operaciones rápidas.

- **Verificar conectividad:**

```
salt '*' test.ping
```

- Devuelve True para cada Minion que responde.

- **Ejecutar un comando en un Minion específico:**

```
salt 'odoo-minion-1' cmd.run 'docker ps | grep odoo'
```

- Esto listaría los contenedores Odoo en ejecución dentro del Minion (si el Minion fuera el host Docker, pero en nuestro laboratorio los Minions son los propios contenedores). Para nuestro caso, un ejemplo más real sería el siguiente para listar las llaves aceptadas desde el propio Master.

```
salt 'ud05-salt-master' cmd.run 'salt-key -L'
```

- **Reiniciar un servicio Odoo (si estuviera gestionado por systemd dentro del contenedor):**

```
salt 'odoo-minion-1' cmd.run 'systemctl restart odoo'
```

- (En el laboratorio, Odoo corre como proceso en primer plano, por lo que este

comando no aplica; pero ilustra el uso de `cmd.run` para tareas de administración.)

- **Automatización desatendida:** Los Minions pueden recibir órdenes para ejecutar comandos CLI que inicializan bases de datos o configuran aplicaciones, evitando la intervención humana. En nuestro laboratorio, la creación de la base de datos de Odoo se realiza mediante un script externo (**04-create-odoo-database.sh**) que usa la API JSON-RPC, no mediante Salt. Por tanto, el uso de Salt en este laboratorio es mínimo (solo el Master, sin states aplicados a Odoo). Sin embargo, para un despliegue completo usando salt, podríamos crear un estado que ejecute:

```
crear_bd_odoo:
  cmd.run:
    - name: |
        curl -X POST http://172.28.0.57:8069/jsonrpc \
          -H "Content-Type: application/json" \
          -d
        '{"jsonrpc": "2.0", "method": "call", "params": {"service": "db", "method": "create_database", "args": ["admin", "odoo_db", true, "es_ES", "admin", "admin", "ES"]}}'
    - unless: curl -s http://172.28.0.57:8069/web/database/list | grep odoo_db
```

- De esta forma, Salt comprobaría si la BD ya existe y, si no, la crearía automáticamente (idempotencia).

Advertencia de seguridad

En el laboratorio, todos los contenedores (incluyendo Salt Master) se ejecutan como root por simplicidad. En un entorno real, esto viola el principio de mínimo privilegio. Para producción, se deben crear usuarios dedicados y usar sudo de forma controlada.

Ejemplo práctico:

Si quisiéramos instalar el paquete `python3-pip` en todos los Minions con Ubuntu, el estado **`install_pip.sls`** sería:

```
install_python_pip:
  pkg.installed:
    - name: python3-pip
    - require:
    - grains: os_family == 'Debian'
```

Y se aplicaría con

```
salt '*' state.apply install_pip
```

Además de los estados, puedes ejecutar comandos ad-hoc con:

```
# Comprobar conectividad del Minion
salt '*' test.ping

# Ejecutar un comando remoto en un Minion concreto
salt 'ud05-salt-master' cmd.run 'docker ps'
```

```
# Instalar una dependencia directamente (sin estado)
salt 'odoo-minion-1' cmd.run 'pip3 install prometheus-client'
```

5. OBSERVANDO EL SISTEMA ERP ODOO

Un sistema ERP que no se monitorea es un sistema ciego. La observabilidad moderna trasciende la simple monitorización de luces verdes y rojas, construyéndose sobre la tríada de métricas, logs y trazas. Para no sobrecargar un solo agente ni afectar al rendimiento del ERP, utilizamos una filosofía de "Separación de Responsabilidades".

Nota sobre versiones del laboratorio

El ecosistema completo incluye 11 contenedores con Loki y Alloy para logs centralizados. No obstante, se proporciona una variante de "bajos recursos" (sin Loki/Alloy y con un solo Odoo) para entornos con ≤ 8 GB de RAM. En esa variante, los logs se consultan directamente con docker logs. Los principios de métricas y alertas son comunes a ambas versiones.

5.1 El puente entre infraestructura y negocio (Whitebox vs. Blackbox)

- **Whitebox (caja blanca) – métricas de negocio:** Aquí es donde el rol DevOps aporta un valor diferencial a la empresa. En lugar de medir solo el consumo de CPU, utilizamos un script personalizado (*odoo-exporter.py*) que se conecta internamente a la API XML-RPC de Odoo. Extrae métricas de negocio en tiempo real (usuarios conectados, facturas emitidas, pedidos de ventas) y las traduce al formato que Prometheus entiende.
 - **Ejemplo de métrica:** *odoo_users_total* (número de usuarios registrados).
- **Blackbox (caja negra) – disponibilidad externa:** Prueba el sistema desde el exterior simulando ser un cliente. Con el *blackbox-exporter* verificamos si la URL de Odoo responde correctamente (código HTTP 200/302) o si hay latencia de red, garantizando la disponibilidad desde la perspectiva del usuario final.
 - **Métrica clave:** *probe_success{instance="http://172.28.0.57:8069"}* vale 1 si Odoo es accesible, 0 si no.

5.2 El stack tecnológico de métricas

- **Prometheus (el motor):** Es una base de datos de series temporales (TSDB) optimizada para guardar millones de puntos de datos. Funciona bajo el modelo "Pull": raspa (scrapea) las métricas navegando por los endpoints de cada servicio cada 15 o 20 segundos (intervalo configurable).
 - **Limitación:** Por defecto, Prometheus retiene los datos solo 15 días. Para periodos más largos es necesario configurar retención explícita (*--storage.tsdb.retention.time*) o integrar con sistemas de almacenamiento a largo plazo.
- **Grafana (la visualización):** Cuadro de mando que consolida las métricas en paneles interactivos. Permite correlacionar, por ejemplo, un pico de estrés en la base de datos con un evento de negocio masivo.
- **Sensores de hardware y contenedores:**

- **Node Exporter:** Lee el estado del sistema operativo anfitrión (CPU, memoria, disco, red).
- **cAdvisor:** Se engancha a los contenedores Docker y disecciona el consumo exacto de recursos de cada contenedor de forma individual.
- **Métrica recomendada para memoria:** *container_memory_working_set_bytes* (representa mejor la memoria activa que *container_memory_usage_bytes*, evitando incluir cachés inactivas).

Ejemplos de consultas PromQL útiles en el laboratorio:

```
# Porcentaje de uso de CPU por contenedor Odoo
rate(container_cpu_usage_seconds_total{name="odoo-minion-1"}[5m]) * 100

# Memoria real usada por Odoo (working set)
container_memory_working_set_bytes{name="odoo-minion-1"}

# Latencia de las consultas a la base de datos Odoo
odoo_query_duration_seconds

# Verificación de que el exporter de Odoo está activo
up{job="odoo-exporter"}
```

5.3 Logs centralizados: la magia de Alloy y Loki (solo en la versión completa)

Revisar registros ejecutando `docker logs` en cada contenedor por separado es inviable en arquitecturas distribuidas.

- **Loki:** Motor de almacenamiento de logs. A diferencia de soluciones pesadas como Elasticsearch, Loki no indexa el texto completo del log, solo sus etiquetas (labels). Esto lo hace extremadamente rápido y eficiente en recursos, ideal para entornos con limitaciones de RAM.
- **Alloy (el agregador):** Agente recolector de telemetría de nueva generación de Grafana (heredero del antiguo Promtail). Se despliega montando el volumen del socket de Docker (`/var/run/docker.sock`). Alloy escucha la salida estándar de todos los contenedores de la infraestructura a la vez, extrae el nombre del contenedor, lo añade como etiqueta (service o container_name) y envía el flujo de texto hacia Loki de forma continua.
 - **Ventaja:** No es necesario instalar agentes dentro de cada contenedor; Alloy actúa desde fuera.

Ejemplo de consulta LogQL en Grafana Explore:

```
logql
{service="odoo-minion-1"} |= "ERROR"
```

Muestra todas las líneas de log del contenedor `odoo-minion-1` que contienen la palabra "ERROR".

⚠ Nota para la versión de bajos recursos:

Si decides no desplegar Loki/Alloy, puedes seguir consultando logs con `docker logs`

odoo-minion-1 o usar el panel de logs de Docker en Grafana (si configuras el datasource de Loki como opcional). La sección 5.3 es aplicable solo al despliegue completo de 11 contenedores.

6. MÉTRICAS CRÍTICAS Y CULTURA DE FIABILIDAD (SRE)

Bajo la filosofía **Site Reliability Engineering (SRE)**, aceptamos que los sistemas van a fallar. En lugar de confiar en la suerte, definimos indicadores de nivel de servicio (SLI) que nos permiten medir la salud del sistema en todo momento. Si un SLI cruza un umbral, se activa una alerta y el equipo puede intervenir antes de que el usuario note el problema. Esta cultura fomenta las postmortems sin culpa (blameless postmortems): aprender del error técnico sin buscar responsables humanos.

En nuestro ecosistema Odoo, estos son los SLI más importantes que debes vigilar desde el cuadro de mando de Grafana:

6.1 Disponibilidad del servicio web (probe_success)

- Qué mide: Si la URL de Odoo responde correctamente (HTTP 200/302) desde el punto de vista de un cliente externo.
- Gestionado por: Blackbox Exporter (sondeo activo).
- Umbral crítico: probe_success == 0 durante más de 1 minuto → indica que el servicio web es inaccesible.
- Alerta: Debe disparar una notificación inmediata (email, Telegram, etc.) porque los usuarios no pueden trabajar.

6.2 Métricas de adopción del negocio (odoo_users_total, odoo_sales_orders_total)

- Qué miden: Número de usuarios registrados y pedidos de venta creados.
- Por qué son importantes: Si la infraestructura está sana (CPU/RAM en verde) pero las ventas caen a cero de golpe, el problema está en la lógica de negocio (un módulo roto, pasarela de pago caída) o en un error de integración.
- Umbral sugerido: Una caída sostenida del 80% respecto a la media histórica durante 30 minutos.

6.3 Latencia de las consultas a la base de datos (odoo_query_duration_seconds)

- Qué mide: Tiempo que tarda Odoo en ejecutar las consultas SQL y devolver el resultado (extraído vía XML-RPC).
- Impacto: Si supera los 500 ms de forma constante, los usuarios percibirán lentitud en formularios e informes.
- Causa posible: Falta de índices en PostgreSQL, tablas muy grandes, o workers insuficientes.
- Nota: La métrica se llama odoo_query_duration_seconds (no odoo_database_query_duration_seconds). Asegúrate de que el exporter use el nombre correcto.

6.4 Saturación de memoria en contenedores (container_memory_working_set_bytes)

- Qué mide: Memoria activa real que está usando un contenedor (excluye cachés inactivas). Se obtiene de cAdvisor.
- Umbral crítico: Cuando supera el 90% del límite asignado (ej. 1024 MB → 921 MB), el OOM Killer está a punto de matar el contenedor.
- Acción: Revisar fugas de memoria, aumentar el límite o escalar horizontalmente.
- Recomendación: Usar container_memory_working_set_bytes en lugar de container_memory_usage_bytes (este último infla el consumo incluyendo cachés).

7. ★ BUENAS PRÁCTICAS EN AUTOMATIZACIÓN Y MONITORIZACIÓN

Para que infraestructuras como la que hemos construido no se vuelvan frágiles, el diseño debe obedecer a principios arquitectónicos estrictos. A continuación se enumeran las prácticas esenciales, muchas de ellas ya aplicadas (y otras que deberías considerar en entornos reales).

7.1 Idempotencia: ejecuta el despliegue tantas veces como quieras

- **Definición:** Un script o estado es idempotente si, al ejecutarlo varias veces seguidas, el sistema acaba siempre en el mismo estado final (sin errores ni efectos secundarios).
 - Los archivos de Terraform son idempotentes: al volver a aplicar terraform apply, solo se modifican los recursos que han cambiado.
 - Los scripts del laboratorio deben intentar ser idempotentes o al menos parcialmente idempotentes.

7.2 Mitigación de la fatiga de alertas (alert fatigue)

- **El problema:** Es un problema de dar demasiadas alertas. Un ejemplo: si el sistema envía un correo cada vez que la CPU supera el 90% durante un segundo, el administrador ignorará los avisos.
 - **Solución:** Configurar ventanas de tiempo en las reglas de Prometheus (cláusula for: 5m) para que la alerta solo se dispare cuando la anomalía se mantenga durante un periodo significativo.

7.3 Desacoplamiento de datos: los volúmenes salvan vidas

- **Principio:** La computación debe ser efímera, pero la historia debe persistir. Nunca se deben almacenar datos de bases de datos o configuraciones de cuadros de mando dentro del sistema de archivos interno de un contenedor.
- **Aplicación en el laboratorio:**
 - PostgreSQL, Prometheus, Grafana y Loki usan volúmenes Docker (ud05_odoo_pgsql_1, ud05_prometheus_data, etc.) mapeados a carpetas del host.
- **Mejora necesaria:** En un contexto real, sería necesario implementar backups periódicos de esos volúmenes (por ejemplo, con ***docker run --rm -v ud05_prometheus_data:/data***

-v \$(pwd):/backup alpine tar czf /backup/prometheus-backup.tar.gz /data).

7.4 Principio de mínimo privilegio: no ejecute como root

- **Situación actual:** Todos los contenedores del laboratorio se ejecutan con user = "root" (por simplicidad). Esto es un riesgo de seguridad porque un atacante que comprometa el contenedor tendrá control total sobre el host.
- **Buenas prácticas:**
 - Crear un usuario específico para cada servicio (ej. odoo, postgres, prometheus) con los permisos mínimos necesarios.
 - En el **startup-odoo.sh**, cambiar a ese usuario antes de ejecutar Odoo (ya se hace con su - odoo).
 - En Terraform, especificar user = "1000:1000" (UID del usuario odoo) en lugar de root.

7.5 Automatización no interactiva (modo desatendido)

- **Definición:** En despliegues masivos, las instalaciones no deben esperar confirmaciones humanas (como apt-get pidiendo [Y/n]).
- **Aplicación en el laboratorio:**
 - Se usa export DEBIAN_FRONTEND=noninteractive al principio de los scripts.
 - Se añade -y a los comandos apt-get install.

7.6 Seguridad en la red y cifrado

- **Problema:** Todos los contenedores de la red LabCEFIRE se comunican en texto plano (sin TLS). Esto es aceptable en un laboratorio, pero no en producción.
- **Recomendación:**
 - Para Odoo, habilitar HTTPS con un proxy inverso (Nginx) y un certificado autofirmado o de Let's Encrypt.
 - Para Prometheus y Grafana, usar autenticación básica o integración con OAuth.

7.7 Monitorización de los propios sistemas de monitorización

- **El riesgo:** Si Prometheus o Grafana fallan, no recibirás alertas.
- **Solución** (fuera del alcance del laboratorio, pero a tener en cuenta):
 - Desplegar un segundo Prometheus (modo federado) que monitoree al primero.
 - Usar un servicio externo (como UptimeRobot) para comprobar que Grafana responde.

7.8 Documentación como código

- **Práctica:** Mantener la documentación (como este mismo documento) actualizada y vinculada al código (README en el repositorio, comentarios en los scripts).
- **En el laboratorio:** Los scripts contienen comentarios explicativos y colores en la salida para facilitar la lectura de logs.

8. WEBGRAFÍA.

- **Odoo Performance Monitoring: Prometheus, Grafana, pg_stat_statements** 
<https://www.braincuber.com/tutorial/monitoring-odoo-performance-essential-metrics-tools>
 - Documentación exhaustiva sobre la monitorización del rendimiento de Odoo, el cálculo óptimo de workers, y la extracción de métricas clave de negocio (Caja Blanca).
- **Salt System Architecture** 
https://docs.saltproject.io/en/3007/topics/salt_system_architecture.html
 - Documentación oficial del ecosistema SaltStack. Vital para entender a bajo nivel el bus de mensajes ZeroMQ, el paradigma Master-Minion y la gestión de Grains y Pillars.
- **Top 10 Terraform Best Practices for Infrastructure Automation** 
https://medium.com/@contact_81356/top-10-terraform-best-practices-for-infrastructure-automation-aa192c57b7aa
 - Compendio de buenas prácticas para la orquestación de Infraestructura como Código (IaC) de forma declarativa, gestión del estado e integraciones con proveedores como Docker.
- **Prometheus Monitoring: Complete Setup & Best Practices** 
<https://medium.com/@rubsan/prometheus-monitoring-complete-setup-best-practices-beginner-friendly-9f58dc54191b>
 - Guía principal sobre la filosofía de monitorización moderna basada en extracción (Pull Model) y la configuración avanzada de bases de datos de series temporales (TSDB).
- **How to Implement Blackbox Monitoring with Prometheus** 
<https://oneuptime.com/blog/post/2026-01-25-prometheus-blackbox-monitoring/view>
 - Guía operativa para configurar sondeos activos (Caja Negra) mediante blackbox-exporter y evaluar la disponibilidad externa de los servicios simulando el comportamiento de un usuario final.